

Optimisation pour l'IA

Notes de cours 1 — Mastère Spécialisé IA de Confiance

Séance 1 : Optimisation continue et linéaire

Nadjib Lazaar

CentraleSupélec - Université Paris-Saclay

lazaar@liscn.fr

Sommaire : (1) formalisme du problème & convexité — (2) descente de gradient & accélération — (3) optimisation continue contrainte, multiplicateurs de Lagrange & conditions KKT — (4) programmation linéaire & algorithme du simplexe — (5) dualité.

1 Problèmes d'optimisation : formalisme

Un problème d'optimisation mathématique (générique) s'écrit :

$$\underset{x}{\text{minimiser}} \quad f_0(x) \quad \text{sous contraintes} \quad f_i(x) \leq b_i, \quad i = 1, \dots, m$$

où $x = [x_1, \dots, x_n]^\top \in \mathbb{R}^n$ est la variable d'optimisation, $f_0 : \mathbb{R}^n \rightarrow \mathbb{R}$ est la fonctionnelle de coût (ou objectif), les $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ sont les fonctionnelles de contrainte et les $b_i \in \mathbb{R}$ les bornes correspondantes. Un point x^* est optimal — ou solution du problème — s'il a la plus petite valeur objectif f_0 parmi tous les points qui satisfont les contraintes (l'ensemble admissible). On écrit $x^* = \operatorname{argmin}_x f_0(x)$.

Les contraintes d'égalité $h_i(x) = c_i$ peuvent toujours s'écrire comme deux inégalités $h_i(x) \leq c_i$ et $-h_i(x) \leq -c_i$, donc la formulation ci-dessus est totalement générale. Un problème de maximisation se ramène à cette forme en minimisant $-f_0$.

1.1 Classifier les problèmes d'optimisation

Les problèmes d'optimisation, dans leur pleine généralité, sont en général intraitables : les problèmes NP-complets (le voyageur de commerce, subset-sum, ...) peuvent être formulés comme des problèmes d'optimisation, tout comme des problèmes encore bien plus difficiles. Une certaine régularité mathématique est donc essentielle pour qu'un problème soit soluble efficacement. Trois grands régimes concernent ce cours :

- **Programmation linéaire** (Séance 1, Partie B) : f_0 et chaque f_i sont affines en x . Soluble exactement et efficacement depuis les années 1940 (simplexe, puis méthodes de points intérieurs).
- **Programmation convexe** (Séance 1, Partie A) : f_0 et chaque f_i sont convexes. Tout minimum local est global ; des algorithmes efficaces existent même en l'absence de forme close.
- **Programmation combinatoire / en nombres entiers** (Séance 2) : certaines ou toutes les variables sont restreintes aux entiers. NP-difficile en général, mais souvent modélisé et résolu avec succès en pratique par séparation-évaluation, coupes, ou programmation par contraintes. La croissance combinatoire est vertigineuse : le voyageur de commerce sur n villes compte $(n-1)!/2$ tournées possibles — 181 440 pour 10 villes, déjà plus de 60 millions de milliards pour 20 villes — l'énumération exhaustive est donc hors de portée dès une taille modeste.

Remarque clé : exiger des solutions entières rend généralement — mais pas toujours — un problème beaucoup plus difficile. Certains problèmes structurés (transport, affectation, flot maximal — Séance 3) restent faciles même avec l'intégralité, car leur matrice de contraintes a une structure particulière (nécessairement unimodulaire).

1.2 Fonctions convexes

Une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est convexe si son domaine est convexe et si, pour tous x_1, x_2 de son domaine et tout $\lambda \in [0, 1]$:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2).$$

Géométriquement : la corde joignant deux points quelconques du graphe de f reste sur ou au-dessus du graphe. Remplacer $\leq$ par $=$ dans la définition donne la définition d'une fonction affine (ú linéaire ž) — les fonctions affines sont le cas particulier à la fois convexe et concave.

1.2.1 Pourquoi la convexité compte

- Tout minimum local d'une fonction convexe est aussi un minimum global — aucun risque de rester ú coincé ž dans un mauvais optimum local.
- Pour une f convexe différentiable, x^* est un minimiseur global si et seulement si $\nabla f(x^*) = 0$ (cas non contraint) — l'information au premier ordre suffit à certifier l'optimalité.
- Les sommes de fonctions convexes, et les maxima ponctuels de fonctions convexes, sont convexes — cela permet de construire des modèles convexes complexes à partir de briques convexes simples (normes, carrés, exponentielles, ...).

La plupart des fonctions de perte utilisées pour entraîner des modèles de ML classiques (moindres carrés, régression logistique, perte charnière du SVM, pénalités ridge/LASSO) sont convexes. Les pertes des réseaux de neurones profonds ne sont en général pas convexes, ce qui explique pourquoi leur entraînement n'offre de garantie que d'atteindre un minimum local — souvent ú suffisamment bon ž.

1.3 Un style de programmation : déclaratif plutôt qu'impératif

Écrire ú minimiser $f_0(x)$ sous contraintes $f_i(x) \leq b_i$ ž n'est pas un programme au sens habituel du terme — ce n'est pas une suite d'instructions qui calcule un résultat étape par étape. C'est une description *déclarative* du problème : on énonce les propriétés que doit satisfaire la solution (les variables, l'objectif, les contraintes), sans préciser comment la trouver. C'est au solveur — simplexe, descente de gradient, CP-SAT, ... — de déterminer le ú comment ž.

Cela s'oppose à la programmation *impérative* (ou procédurale), le style Python habituel : une boucle qui met à jour w à chaque itération pour la descente de gradient est un programme impératif — vous décrivez explicitement le calcul à effectuer, pas seulement le résultat voulu. D'autres langages ou formalismes déclaratifs familiers suivent la même logique : une requête SQL décrit quelles lignes on veut, pas l'algorithme de jointure qui les trouve ; une formule de tableur décrit une relation entre cellules, pas l'ordre de recalcul.

La programmation linéaire (ce cours) et la programmation par contraintes ou PPC (Séance 2) sont deux grandes familles de programmation déclarative dédiées à l'optimisation et à la satisfaction de contraintes. Dans les deux cas, vous écrivez le modèle — variables, objectif et/ou contraintes — et un solveur générique se charge de trouver une solution, ou de prouver qu'il n'en existe pas.

Style	Ce que vous écrivez	Exemple dans ce cours
Impératif / procédural	La suite d'instructions qui calcule le résultat, pas à pas	La boucle de descente de gradient (Partie A)
Déclaratif — programmation linéaire	Des variables, un objectif linéaire, des contraintes linéaires	Le mix de production ExedBike (Partie B)
Déclaratif — programmation par contraintes	Des variables, leurs domaines, des contraintes (souvent combinatoires)	Ordonnancement, Sudoku (Séance 2)

Pourquoi ça compte

Le solveur résout le problème comment ça veut pour vous — mais seulement si le modèle qu'on lui donne (le problème) est correct et complet. Une grande partie de la compétence en optimisation n'est donc pas de coder un algorithme (vous ne réécrivez pas le simplexe à chaque projet), mais d'écrire un modèle juste : les bonnes variables, le bon objectif, toutes les bonnes contraintes. C'est exactement l'objet de la Partie 1 de ce cours, et vous retrouverez ce même réflexe de modélisation déclarative avec la PPC en Séance 2.

2 Méthodes itératives pour les problèmes lisses

2.1 Descente de gradient

Considérons le problème non contraint minimiser $J(w)$, $w \in \mathbb{R}^n$, avec J différentiable. La descente de gradient génère une suite d'itérés en se déplaçant dans la direction de plus forte décroissance locale de J :

$$w_{k+1} = w_k - \alpha \nabla J(w_k).$$

Le scalaire $\alpha > 0$ est le pas (ou *learning rate*). Intuition : $\nabla J(w_k)$ pointe dans la direction de plus forte augmentation de J en w_k , donc se déplacer dans la direction opposée diminue J , au moins pour α assez petit (un développement de Taylor au premier ordre de J autour de w_k le montre précisément : $J(w_k - \alpha \nabla J(w_k)) \approx J(w_k) - \alpha \|\nabla J(w_k)\|^2 < J(w_k)$ pour α petit et $\nabla J(w_k) \neq 0$).

2.1.1 Convergence et rôle du pas

Pour J convexe et L -lisse (∇J est L -lipschitzienne), la descente de gradient à pas fixe $\alpha \leq 1/L$ converge, et pour $\alpha = 1/L$ l'écart d'objectif vérifie $J(w_k) - J(w^*) = O(1/k)$: un taux sous-linéaire. Si J est de plus m -fortement convexe, la convergence devient linéaire (géométrique) : $\|w_k - w^*\|^2 \leq (1 - m\alpha)^k \|w_0 - w^*\|^2$, approximativement.

En pratique, le pas ne peut être ni trop grand (risque de divergence ou d'oscillation) ni trop petit (convergence douloureusement lente). Deux remèdes standards :

- Pas fixe issu de la théorie : $\alpha = 1/L$ quand L est connu ou estimable (ex. L = plus grande valeur propre de la Hessienne pour un quadratique).
- Recherche linéaire par rebroussement (Armijo) : à chaque itération, on essaie un pas et on le réduit géométriquement jusqu'à ce qu'une condition de décroissance suffisante soit vérifiée — s'adapte automatiquement, au prix d'évaluations supplémentaires de la fonction.

2.1.2 Mauvais conditionnement

Considérons le quadratique $J(w) = \frac{1}{2} w^\top A w$ avec A symétrique définie positive. La vitesse de convergence de la descente de gradient dépend du conditionnement $\kappa(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$: plus κ est grand, plus les lignes de niveau de J sont des ellipses allongées, et plus la descente de gradient zigzague selon la direction étroite au lieu de progresser selon la direction longue. C'est exactement le phénomène que vous observerez au TP1, Exercice 1, et qui motive les techniques d'accélération ci-dessous (et, plus tard, les méthodes adaptatives par paramètre comme Adam).

2.2 Méthodes accélérées

2.2.1 Momentum (heavy-ball)

$$v_{k+1} = \beta v_k - \alpha \nabla J(w_k), \quad w_{k+1} = w_k + v_{k+1}.$$

Un terme de vitesse v_k accumule une moyenne mobile (pondérée exponentiellement) des gradients passés ; $\beta \in [0, 1)$ est le coefficient de momentum (valeurs typiques 0,8–0,99). Cela amortit les oscillations selon les directions étroites tout en renforçant la progression selon les directions constamment descendantes.

2.2.2 Gradient accéléré de Nesterov

$$v_{k+1} = \beta v_k - \alpha \nabla J(w_k + \beta v_k), \quad w_{k+1} = w_k + v_{k+1}.$$

La seule différence avec le momentum est l'endroit où le gradient est évalué : au point $\hat{z}$ anticipé $\hat{z} = w_k + \beta v_k$ plutôt qu'en w_k lui-même. Ce changement, en apparence mineur, donne un taux de convergence $O(1/k^2)$ optimal pour les problèmes convexes lisses (contre $O(1/k)$ pour la descente de gradient simple) — un résultat célèbre dû à Nesterov (1983).

2.2.3 Autres méthodes (aperçu rapide)

- **Gradient conjugué** : pour les quadratiques, génère des directions de recherche mutuellement $\hat{z}$ conjuguées $\hat{z}$ par rapport à A , garantissant une convergence exacte en au plus n pas (en arithmétique exacte) ; s'étend heuristiquement aux problèmes lisses non quadratiques.
- **BFGS (quasi-Newton)** : construit une approximation de la Hessienne (inverse) à partir des gradients successifs, approximant la méthode de Newton sans jamais former la vraie Hessienne.
- **Adam / RMSProp** : maintiennent des pas adaptatifs par paramètre à partir d'estimations glissantes des premier et second moments du gradient — le choix par défaut pour entraîner de grands réseaux de neurones (voir le module *Développement Python pour l'apprentissage*).

3 Optimisation continue sous contraintes

3.1 Gérer les contraintes avec les méthodes de gradient

- **Descente de gradient projetée** : $w_{k+1} = \text{Proj}_{\mathcal{C}}(w_k - \alpha \nabla J(w_k))$, où $\text{Proj}_{\mathcal{C}}$ projette sur l'ensemble admissible après un pas non contraint. Simple et exacte quand la projection est peu coûteuse (contraintes de boîte, simplexe, boule ℓ_2).
- **Méthodes de pénalité** : on minimise $J(x) + \rho \cdot \text{pénalité}(x)$, où $\text{pénalité}(x)$ croît quand les contraintes sont violées (ex. $\text{pénalité}(x) = \sum \max(0, g_i(x))^2$). Quand $\rho \rightarrow \infty$, le minimiseur du problème pénalisé s'approche de l'optimum contraint, au prix d'un mauvais conditionnement croissant.
- **Méthodes de barrière (points intérieurs)** : pour des contraintes $g_i(x) \geq 0$, on minimise $J(x) - \xi \sum \log(g_i(x))$, qui n'est finie que strictement à l'intérieur de l'ensemble admissible (une $\hat{z}$ barrière $\hat{z}$ contre toute sortie). Résoudre une suite de tels problèmes avec $\xi \rightarrow 0$ trace le $\hat{z}$ chemin central $\hat{z}$ vers le véritable optimum contraint tout en gardant chaque itéré admissible — c'est la base des solveurs modernes de points intérieurs pour le PL et l'optimisation convexe.

Exemple travaillé : pénalité vs. barrière

minimiser $f(x) = (x+1)^2$ s.c. $x \geq 0$. Le minimum non contraint est $x = -1$, infaisable.
Pénalité $f(x) + \rho x^2$ (pour $x < 0$) : le minimiseur est $x_\rho = -1/(1+\rho)$, qui se rapproche de 0 (la frontière admissible) quand $\rho \rightarrow \infty$ mais reste toujours légèrement infaisable. Pour $\rho = 2$: $x^* \approx -0,333$. Pour $\rho = 20$: $x^* \approx -0,0476$ — on se rapproche, mais le point reste hors de l'ensemble admissible.
Barrière $f(x) - \xi \log(x)$ (pour $x > 0$) : le minimiseur est $x_\xi = \frac{-1+\sqrt{1+2\xi}}{2}$, toujours strictement admissible. Pour $\xi = 1,2$: $x^* \approx 0,422$. Pour $\xi = 0,3$: $x^* \approx 0,1325$. Pour $\xi = 0,03$: $x^* \approx 0,0148$ — convergence vers $x = 0$ en restant toujours du bon côté de la contrainte, contrairement à la pénalité.

Vous implémenterez et comparerez une méthode de barrière, le SLSQP de scipy (un solveur contraint direct) et cvxpy (une couche de modélisation pour les problèmes convexes) sur le même exemple au TP1, Exercice 3.

3.2 Multiplicateurs de Lagrange et conditions KKT

Considérons le problème contraint

$$\underset{x}{\text{minimiser}} \quad f_0(x) \quad \text{s.c.} \quad g_i(x) \geq 0, \quad i = 1, \dots, m$$

(les contraintes d'égalité peuvent s'écrire comme deux inégalités opposées, ou être gérées avec leurs propres multiplicateurs λ_j — omis ici par souci de concision). Le lagrangien est

$$L(x, \mu) = f_0(x) - \sum_i \mu_i g_i(x).$$

Les conditions de Karush–Kuhn–Tucker (KKT) sont nécessaires pour que x^* soit optimal (et, pour les problèmes convexes avec une qualification de contrainte, également suffisantes) :

- **Stationnarité** : $\nabla f_0(x^*) = \sum_i \mu_i \nabla g_i(x^*)$
- **Admissibilité primale** : $g_i(x^*) \geq 0$ pour tout i
- **Admissibilité duale** : $\mu_i \geq 0$ pour tout i
- **Complémentarité** : $\mu_i g_i(x^*) = 0$ pour tout i (donc $\mu_i = 0$ sauf si la contrainte i est active / saturée)

Interprétation de la stationnarité : à l'optimum, le gradient du coût ne peut être qu'une combinaison non négative des gradients des contraintes actives — sinon il existerait une direction admissible le long de laquelle f_0 continuerait de décroître, contredisant l'optimalité. Quand plusieurs contraintes sont simultanément actives (un n sommet z de l'ensemble admissible, exactement comme un sommet d'un PL), le système comporte un multiplicateur par contrainte active et est génériquement bien posé. L'exemple ci-dessous n'a qu'une seule contrainte active à l'optimum — le cas le plus simple, mais qui illustre déjà toute la mécanique du système KKT ; vous résoudrez un système de ce type au TP1, Exercice 3, et le vérifierez numériquement avec le SLSQP de scipy.

Exemple travaillé

minimiser $f_0(x, y) = (x - 2)^2 + (y - 2)^2$ s.c. $g(x, y) = 2 - x - y \geq 0$. Le minimum non contraint $(2, 2)$ est infaisable ($g(2, 2) = -2 < 0$), donc la contrainte est active à l'optimum. On a $\nabla f_0(x, y) = (2x - 4, 2y - 4)$ et $\nabla g(x, y) = (-1, -1)$ (constant). La stationnarité $\nabla f_0(x^*, y^*) = \mu \nabla g(x^*, y^*)$ impose que ∇f_0 soit colinéaire à $(-1, -1)$, donc $2x^* - 4 = 2y^* - 4$, soit $x^* = y^*$; combiné à la contrainte active $x^* + y^* = 2$, cela donne $x^* = y^* = 1$. Vérification : $\nabla f_0(1, 1) = (-2, -2) = 2 \cdot (-1, -1) = \mu \nabla g(1, 1)$ avec $\mu = 2 \geq 0$ — KKT est satisfait. Interprétation géométrique : la ligne de niveau $f_0 = 2$ (un cercle centré en $(2, 2)$) est tangente à la droite $x + y = 2$ exactement en $x^* = (1, 1)$; résultat confirmé indépendamment par le SLSQP de scipy.

4 Programmation linéaire

4.1 Trois formes d'un programme linéaire

Un même programme linéaire (PL) peut s'écrire sous plusieurs formes équivalentes, chacune utile pour une étape différente : la forme la plus naturelle pour modéliser, la forme canonique pour construire le dual, la forme standard pour appliquer l'algorithme du simplexe.

- **Forme générale** : n'importe quel mélange de contraintes $\leq, \geq, =$, et des variables de signe quelconque (positives, négatives ou libres). C'est la forme naturelle issue de la modélisation d'un problème métier.
- **Forme canonique** : maximiser $c^\top x$ s.c. $Ax \leq b, x \geq 0$. Toutes les contraintes sont des inégalités $\leq$, toutes les variables sont non négatives.
- **Forme standard** : maximiser $c^\top x$ s.c. $Ax = b, x \geq 0$. Toutes les contraintes sont des égalités, toutes les variables sont non négatives — c'est la forme sur laquelle se construit directement le tableau du simplexe (§4.3).

Règles de conversion, de la forme générale vers la forme canonique :

- minimiser $f \implies$ maximiser $-f$ (même solution, valeur optimale changée de signe) ;
- contrainte $\geq \implies$ multiplier les deux membres par -1 pour obtenir un $\leq$;
- contrainte d'égalité $h(x) = c \implies$ remplacer par deux inégalités $h(x) \leq c$ et $-h(x) \leq -c$;
- variable libre $x \in \mathbb{R} \implies$ remplacer par $x = x^+ - x^-$ avec $x^+, x^- \geq 0$.

De la forme canonique vers la forme standard : ajouter une variable d'écart non négative par contrainte $\leq$ (voir §4.3).

Pourquoi ces transformations ?

Chaque forme sert un usage précis. La forme canonique est celle sur laquelle la construction du dual (§4.4) est la plus directe — la *ré* recette *z* primal $\rightarrow$ dual s'énonce proprement pour $Ax \leq b, x \geq 0$. La forme standard est celle sur laquelle le tableau du simplexe se lit directement (§4.3) : chaque égalité devient une ligne du tableau, et la solution de base admissible initiale s'obtient *ré* gratuitement *z* en posant les variables structurelles à zéro. Passer systématiquement par ces deux formes, plutôt que d'adapter l'algorithme à chaque cas particulier, est ce qui permet d'implémenter le simplexe une seule fois pour tous les PL possibles, quelle que soit leur écriture de départ.

Exemple travaillé

minimiser $z = 2x_1 - 3x_2$ s.c. $x_1 + x_2 \geq 4, 2x_1 - x_2 = 6, x_1 \geq 0, x_2$ libre ($\in \mathbb{R}$).

Forme canonique (maximiser $w = -z; x_2 = x_2^+ - x_2^-$ avec $x_2^+, x_2^- \geq 0$) :

$$\begin{aligned} \underset{x_1, x_2^+, x_2^-}{\text{maximiser}} \quad & -2x_1 + 3x_2^+ - 3x_2^- \\ \text{s.c.} \quad & -x_1 - x_2^+ + x_2^- \leq -4 \\ & 2x_1 - x_2^+ + x_2^- \leq 6 \\ & -2x_1 + x_2^+ - x_2^- \leq -6 \\ & x_1, x_2^+, x_2^- \geq 0 \end{aligned}$$

Forme standard (ajout des écarts $s_1, s_2, s_3 \geq 0$) :

$$\begin{aligned} \underset{x_1, x_2^+, x_2^-, s_1, s_2, s_3}{\text{maximiser}} \quad & -2x_1 + 3x_2^+ - 3x_2^- \\ \text{s.c.} \quad & -x_1 - x_2^+ + x_2^- + s_1 = -4 \\ & 2x_1 - x_2^+ + x_2^- + s_2 = 6 \\ & -2x_1 + x_2^+ - x_2^- + s_3 = -6 \\ & x_1, x_2^+, x_2^-, s_1, s_2, s_3 \geq 0 \end{aligned}$$

Vous pratiquerez cette conversion générale $\rightarrow$ canonique $\rightarrow$ standard sur un exemple complet en TD1 (Exercice 2, Question 5).

4.2 Géométrie du polytope

Reprenons la forme canonique introduite ci-dessus (écrite ici en minimisation, ce qui ne change rien à la géométrie) :

$$\underset{x}{\text{minimiser}} \quad c^\top x \quad \text{s.c.} \quad Ax \leq b, \quad x \geq 0,$$

avec $c \in \mathbb{R}^n$ le vecteur de coûts, $A \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$ décrivant les contraintes. L'objectif et chaque contrainte sont affines — aucun produit ni puissance de variables.

Chaque inégalité linéaire définit un demi-espace ; l'intersection d'un nombre fini de demi-espaces est un polyèdre convexe (un polytope, s'il est borné). Un fait fondamental de la programmation linéaire est :

Théorème fondamental du PL

Si un programme linéaire admet une solution optimale, alors il en admet une en un sommet (point extrême) du polytope admissible. C'est ce qui rend le PL fondamentalement plus facile que l'optimisation convexe générale : au lieu de chercher dans un continuum, un algorithme optimal n'a besoin d'examiner qu'un ensemble fini (bien que potentiellement grand) de sommets.

4.3 L'algorithme du simplexe (G. Dantzig, 1947)

L'algorithme du simplexe exploite directement le théorème fondamental : il se déplace de sommet en sommet adjacent du polytope, en choisissant à chaque étape une arête qui améliore (ou, dans les cas dégénérés, ne dégrade pas) l'objectif, jusqu'à ce qu'aucune arête améliorante n'existe — le sommet courant est alors optimal.

4.3.1 Du système standard au tableau initial

Pour appliquer la méthode du tableau, les contraintes d'inégalité sont d'abord converties en égalités en ajoutant des variables d'écart non négatives : $Ax \leq b$ devient $Ax + s = b, s \geq 0$ — c'est exactement la forme standard de la §4.1. Une solution de base admissible fixe n des $(n + m)$ variables à zéro (í hors base ž) et résout pour les m variables restantes (í de base ž).

Reprenons un exemple canonique concret — une petite menuiserie fabrique des tables (t) et des chaises (c) ; chaque table rapporte 70 et utilise 4 planches de bois et 2 heures-machine, chaque chaise rapporte 50 et utilise 3 planches et 1 heure-machine ; l'atelier dispose de 240 planches et 100 heures-machine par semaine :

$$\underset{t,c}{\text{maximiser}} \quad 70t + 50c \quad \text{s.c.} \quad 4t + 3c \leq 240 \text{ (bois)}, \quad 2t + c \leq 100 \text{ (machine)}, \quad t, c \geq 0.$$

Forme standard (écart s_1 pour le bois, s_2 pour la machine) :

$$\begin{aligned} 4t + 3c + s_1 &= 240 \\ 2t + c + s_2 &= 100 \\ t, c, s_1, s_2 &\geq 0 \end{aligned}$$

On en tire le tableau initial directement, sans aucun calcul :

- chaque équation du système standard devient une ligne du tableau (une ligne par variable de base) ;
- poser $t = c = 0$ (variables structurelles hors base) donne immédiatement une solution de base admissible : $s_1 = 240, s_2 = 100$ — í gratuitement ž, sans rien résoudre ;
- la ligne Z du tableau porte les coefficients de l'objectif changés de signe, puisqu'on l'écrit $Z - 70t - 50c = 0$.

4.3.2 Exemple travaillé pas à pas : la menuiserie

Tableau 1 (initial) — variables de base s_1, s_2 :

	t	c	s_1	s_2	RHS
s_1	4	3	1	0	240
s_2	2	1	0	1	100
Z	-70	-50	0	0	0

Le coût réduit le plus négatif est celui de t (-70) : t entre en base. Test du rapport : $\min(240/4, 100/2) = \min(60, 50) = 50 \rightarrow s_2$ sort (pivot = 2, ligne s_2).

Tableau 2 (après un pivot) — variables de base s_1, t :

	t	c	s_1	s_2	RHS
s_1	0	1	1	-2	40
t	1	0,5	0	0,5	50
Z	0	-15	0	35	3500

Le coût réduit de c est encore négatif (-15) : c entre en base. Test du rapport : $\min(40/1, 50/0,5) = \min(40, 100) = 40 \rightarrow s_1$ sort (pivot = 1, ligne s_1).

Tableau 3 (optimal) — variables de base c, t :

	t	c	s_1	s_2	RHS
c	0	1	1	-2	40
t	1	0	-0,5	1,5	30
Z	0	0	15	5	4100

Tous les coûts réduits de la ligne Z sont désormais non négatifs : le tableau est optimal. Solution : $t^* = 30$, $c^* = 40$, $s_1 = s_2 = 0$ (bois et machine tous deux saturés), objectif optimal $Z^* = 70 \times 30 + 50 \times 40 = 2100 + 2000 = 4100$ par semaine — confirmé indépendamment par `scipy.optimize.linprog`. On retrouve d'ailleurs le même sommet par énumération directe des sommets du polytope (méthode géométrique, TD1).

4.3.3 Application : le mix de production ExedBike

Le même modèle s'applique directement au problème du mix de production ExedBike développé en Partie 1 (TD1) : maximiser $120u + 200c$ s.c. $4u + 6c \leq 480$ (main-d'œuvre), $6u + 15c \leq 900$ (aluminium), $2u + 2c \leq 240$ (temps machine), $u, c \geq 0$. En appliquant exactement la même méthode — mise en forme standard puis pivots successifs — on obtient le sommet optimal $u^* = 75$, $c^* = 30$, avec la main-d'œuvre et l'aluminium saturés ($s_1 = s_2 = 0$) et une marge de 30 heures sur le temps machine ($s_3 = 30$), pour un objectif optimal $120 \times 75 + 200 \times 30 = 9000 + 6000 = 15\,000$ par semaine. Vous pratiquerez la mise en forme standard sur un exemple complet en TD1 (Exercice 2, Question 5) ; vous reconstruirez la séquence complète de pivots pour ExedBike et la vérifierez avec `scipy.optimize.linprog`, PuLP et le Solveur Excel au TP1 — trois implémentations indépendantes de cet algorithme (ou d'une méthode de points intérieurs apparentée).

4.4 Dualité et prix duaux

Tout PL (le primal) possède un PL dual associé ; pour le PL, la dualité forte est vérifiée : les valeurs optimales primale et duale coïncident (sous des conditions modérées). La variable duale associée à une contrainte primale s'appelle son prix dual : la variation marginale de l'objectif optimal par unité de relâchement du second membre de cette contrainte.

4.4.1 La recette : du primal au dual

Pour un primal sous forme canonique maximiser $c^\top x$ s.c. $Ax \leq b$, $x \geq 0$, le dual s'obtient par une règle mécanique : une variable duale y_i par contrainte primale, une contrainte duale par variable primale, et le sens des inégalités s'inverse :

$$\text{Primal : maximiser } c^\top x \text{ s.c. } Ax \leq b, x \geq 0 \quad \Longleftrightarrow \quad \text{Dual : minimiser } b^\top y \text{ s.c. } A^\top y \geq c, y \geq 0.$$

Exemple travaillé (menuiserie, §4.3). Primal : maximiser $70t + 50c$ s.c. $4t + 3c \leq 240$ (bois), $2t + c \leq 100$ (machine). Dual (une variable y_1 pour la contrainte bois, y_2 pour la contrainte machine ; une contrainte duale par variable primale t et c) :

$$\underset{y_1, y_2}{\text{minimiser}} \quad 240y_1 + 100y_2 \quad \text{s.c.} \quad 4y_1 + 2y_2 \geq 70 \text{ (table)}, \quad 3y_1 + y_2 \geq 50 \text{ (chaise)}, \quad y_1, y_2 \geq 0.$$

Vérification par complémentarité. À l'optimum primal, $t^* = 30 > 0$ et $c^* = 40 > 0$ (aucune des deux variables n'est nulle), donc par complémentarité les deux contraintes duales sont nécessairement saturées : $4y_1 + 2y_2 = 70$ et $3y_1 + y_2 = 50$. Résoudre ce système linéaire 2×2 donne $y_1 = 15$ /planche, $y_2 = 5$ /heure-machine. Vérification de la dualité forte : $240 \times 15 + 100 \times 5 = 3600 + 500 = 4100$ = valeur optimale primale — les deux coïncident, comme attendu.

4.4.2 Retour à ExedBike

Pour l'exemple ExedBike : le prix dual de la main-d'œuvre est 25 /heure (chaque heure de main-d'œuvre supplémentaire augmenterait la marge hebdomadaire de 25, au voisinage de l'optimum courant), celui de l'aluminium 3,33 /kg, et celui du temps machine 0 — il dispose de 30 heures de marge, donc une unité marginale de temps machine ne vaut rien actuellement (cohérent avec la complémentarité : $s_3 = 30 > 0 \Rightarrow$ prix dual = 0). Cela donne à un directeur d'usine un classement précis et actionnable des ressources sur lesquelles investir en priorité.

5 Synthèse et pour aller plus loin

Thème	Idée clé	Où vous l'utiliserez
Convexité	Minimum local = minimum global	Certifier l'optimalité tout au long du cours
Descente de gradient	$w_{k+1} = w_k - \alpha \nabla J(w_k)$	TP1 Ex.1–2 ; ajustement du modèle de coût ExedBike
Momentum / Nesterov	Accumuler une vitesse pour traverser rapidement les vallées étroites	TP1 Ex.2 ; entraînement des réseaux de neurones
Barrière / KKT	Gérer les contraintes en restant (ou en convergeant vers) l'admissibilité	TP1 Ex.3
Programmation linéaire	Optimum en un sommet du polytope ; le simplexe se déplace de sommet en sommet	TP1 Ex.4 ; mix de production ExedBike
Dualité / prix duaux	Les variables duales valorisent chaque contrainte à la marge	Interprétation métier des résultats du PL

Références

- S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004 (ouvrage de référence du cours).
- H. Talbot, *Lecture Notes on Optimisation — Applications to Computer Vision, Image and Signal Processing, and Machine Learning*, CentraleSupélec, 2024 (lecture complémentaire sur les méthodes itératives).
- G. Dantzig, *Linear Programming and Extensions*, Princeton University Press, 1963 (référence historique sur l'algorithme du simplexe).

La suite — Séance 2

Programmes entiers et 0-1, séparation-évaluation, coupes de Gomory, et une première rencontre avec la programmation par contraintes (CP-SAT). Nous nous appuyons directement sur le modèle de mix de production ExedBike développé ici.